

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Schema Analysis and Viva Voce

COURSE NAME: Query Processing for
Data Science
COURSE CODE: DSA0503

COURSE OUTCOME COVERED

CO2: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)

Assessment Tool Weightage: 35%

Dave's Taxonomy: Manipulation (Level 2) Question Count: 2

Total Marks: 20

Code of Conduct

I **GUNA P (192424277)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Database Design and Connectivity Rubric

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

Total Marks Awarded:

Questions:

Question 1: Student Course Registration Database (10 Marks)

Aim

To create Student, Course, and Enrollment tables using SQLite and display the names of students along with the courses they enrolled in during the **Odd 2026** semester using an SQL JOIN query.

Algorithm

1. Import the sqlite3 library.
2. Connect to the SQLite database.
3. Create Student, Course, and Enrollment tables.
4. Insert sample records.
5. Execute an INNER JOIN query.
6. Retrieve students enrolled in **Odd 2026**.
7. Display the results.
8. Close the database connection.

CODE

```
import sqlite3

# Connect to SQLite Database
conn = sqlite3.connect("University.db")
cursor = conn.cursor()

# Create Student Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
    Student_ID INTEGER PRIMARY KEY,
    Student_Name TEXT,
    Department TEXT,
    Year INTEGER
)
""")

# Create Course Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Course(
    Course_ID INTEGER PRIMARY KEY,
    Course_Name TEXT,
    Credits INTEGER
)
""")

# Create Enrollment Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Enrollment(
    Enrollment_ID INTEGER PRIMARY KEY,
```

```

Student_ID INTEGER,
Course_ID INTEGER,
Semester TEXT,
FOREIGN KEY(Student_ID) REFERENCES Student(Student_ID),
FOREIGN KEY(Course_ID) REFERENCES Course(Course_ID)
)
)"""

# Insert Sample Data
cursor.executemany("INSERT OR IGNORE INTO Student VALUES(?,?,?,?)",[
(1,'Arun','CSE',2),
(2,'Priya','AI&DS',3),
(3,'Kavin','ECE',2)
])

cursor.executemany("INSERT OR IGNORE INTO Course VALUES(?,?,?)",[
(101,'Python Programming',4),
(102,'Database Systems',3),
(103,'Artificial Intelligence',4)
])

cursor.executemany("INSERT OR IGNORE INTO Enrollment VALUES(?,?,?,?)",[
(1,1,101,'Odd 2026'),
(2,1,102,'Odd 2026'),
(3,2,103,'Odd 2026'),
(4,3,102,'Even 2026')
])

conn.commit()

# JOIN Query
cursor.execute("""
SELECT Student.Student_Name,
       Course.Course_Name,
       Enrollment.Semester
FROM Enrollment
INNER JOIN Student
ON Enrollment.Student_ID = Student.Student_ID
INNER JOIN Course
ON Enrollment.Course_ID = Course.Course_ID
WHERE Enrollment.Semester='Odd 2026'
""")

print("Students Enrolled in Odd 2026\n")

for row in cursor.fetchall():
    print(row)

conn.close()

```

OUTPUT

Students Enrolled in Odd 2026

```
('Arun', 'Python Programming', 'Odd 2026')
('Arun', 'Database Systems', 'Odd 2026')
('Priya', 'Artificial Intelligence', 'Odd 2026')
|
```

Question 2: Hospital Management Database (10 Marks)

Aim

To create Doctor, Patient, and Appointment tables using SQLite and display patient names, doctor names, and appointment dates scheduled after **1 January 2026** using an SQL JOIN query.

Algorithm

1. Import sqlite3.
2. Connect to SQLite database.
3. Create Doctor, Patient, and Appointment tables.
4. Insert sample records.
5. Execute JOIN query.
6. Display appointments after **2026-01-01**.
7. Close the connection.

CODE

```
import sqlite3

# Connect to SQLite Database
conn = sqlite3.connect("Hospital.db")
cursor = conn.cursor()

# Create Doctor Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Doctor(
    Doctor_ID INTEGER PRIMARY KEY,
    Doctor_Name TEXT,
    Specialization TEXT
)
""")

# Create Patient Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Patient(
    Patient_ID INTEGER PRIMARY KEY,
    Patient_Name TEXT,
    Age INTEGER
)
""")

# Create Appointment Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Appointment(
    Appointment_ID INTEGER PRIMARY KEY,
    Patient_ID INTEGER,
```

```

    Doctor_ID INTEGER,
    Appointment_Date TEXT,
    FOREIGN KEY(Patient_ID) REFERENCES Patient(Patient_ID),
    FOREIGN KEY(Doctor_ID) REFERENCES Doctor(Doctor_ID)
)
""")

# Insert Sample Data
cursor.executemany("INSERT OR IGNORE INTO Doctor VALUES(?,?,?)",[
(1,'Dr. Kumar','Cardiology'),
(2,'Dr. Meena','Neurology'),
(3,'Dr. Ravi','Orthopedics')
])

cursor.executemany("INSERT OR IGNORE INTO Patient VALUES(?,?,?)",[
(101,'Rahul',25),
(102,'Anitha',30),
(103,'Suresh',45)
])

cursor.executemany("INSERT OR IGNORE INTO Appointment VALUES(?,?,?,?)",[
(1,101,1,'2026-02-10'),
(2,102,2,'2026-03-15'),
(3,103,3,'2025-12-20'),
(4,101,2,'2026-05-01')
])

conn.commit()

# JOIN Query
cursor.execute("""
SELECT Patient.Patient_Name,
       Doctor.Doctor_Name,
       Appointment.Appointment_Date
FROM Appointment
INNER JOIN Patient
ON Appointment.Patient_ID = Patient.Patient_ID
INNER JOIN Doctor
ON Appointment.Doctor_ID = Doctor.Doctor_ID
WHERE Appointment.Appointment_Date > '2026-01-01'
""")

print("Appointments After 1 January 2026\n")

for row in cursor.fetchall():
    print(row)

conn.close()

```

OUTPUT

Appointments After 1 January 2026

('Rahul', 'Dr. Kumar', '2026-02-10')
('Anitha', 'Dr. Meena', '2026-03-15')
('Rahul', 'Dr. Meena', '2026-05-01')

Signature of the Course Faculty